

HTML5 Semantic Tags with Examples

Semantic tags clearly describe the purpose of their content to browsers, developers, search engines, and assistive technologies.

1. `<article>` Tag

The `<article>` element represents independent, self-contained content that can be understood or reused separately from the webpage.

Common uses include:

- Blog posts
- News articles
- Forum posts
- Product reviews
- User comments

```
<article>
  <h2>Benefits of Responsive Web Design</h2>
  <p>
    Responsive design allows a website to adapt
    to different screen sizes.
  </p>
  <p>Published by Ashima</p>
</article>
```

Here, the complete post remains meaningful even if it is displayed on another webpage.

2. `<aside>` Tag

The `<aside>` element contains secondary or supplementary information related to the surrounding content. It is not part of the main content flow.

Common uses include:

- Sidebars
- Related links
- Author biographies
- Advertisements
- Additional tips

```
<main>
  <article>
    <h2>Learning HTML5</h2>
    <p>HTML5 provides semantic elements for page structure.</p>
  </article>

  <aside>
```

```
    <h3>Related Topics</h3>
    <ul>
      <li>CSS3</li>
      <li>JavaScript</li>
    </ul>
  </aside>
</main>
```

The article contains the primary information, while the aside provides related topics.

3. `<figure>` Tag

The `<figure>` element groups self-contained content such as an image, diagram, chart, illustration, or code snippet.

```
<figure>
  
</figure>
```

The figure is related to the main content but can usually be moved to another position without changing the meaning of the page.

4. `<figcaption>` Tag

The `<figcaption>` element provides a caption or explanation for content placed inside a `<figure>`.

It must be the first or last child of the `<figure>` element.

```
<figure>
  

  <figcaption>
    Figure 1: Main university campus building
  </figcaption>
</figure>
```

The `<figure>` groups the media, while `<figcaption>` explains it.

5. `<header>` Tag

The `<header>` element represents introductory content for a webpage or a section.

It commonly contains:

- Website name or logo
- Page heading
- Introductory text
- Search facility
- Navigation links

```
<header>
  <h1>Department of Computer Science</h1>
  <p>Learning, innovation and research</p>

  <nav>
    <a href="index.html">Home</a>
    <a href="courses.html">Courses</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

A webpage can contain multiple `<header>` elements—for example, one for the webpage and another inside an article.

6. `<footer>` Tag

The `<footer>` element represents closing or concluding information for a webpage, section, or article.

It may contain:

- Copyright information
- Author information
- Contact details
- Related links
- Privacy-policy links

```
<footer>
  <p>&copy; 2026 ABC University. All rights reserved.</p>
  <p>
    Contact:
    <a href="mailto:info@example.edu">info@example.edu</a>
  </p>
</footer>
```

Like `<header>`, a webpage may contain more than one `<footer>` element.

7. `<main>` Tag

The `<main>` element contains the dominant and most important content of the webpage.

Content repeated across pages, such as site navigation, logos, and copyright information, should normally remain outside `<main>`.

```
<main>
  <h1>Web Development Course</h1>

  <section>
    <h2>Course Overview</h2>
    <p>This course covers HTML, CSS and JavaScript.</p>
  </section>
</main>
```

A webpage should normally have only one visible `<main>` element.

8. `<mark>` Tag

The `<mark>` element highlights text that is relevant or important in the current context.

```
<p>
  The assignment submission deadline is
  <mark>30 August 2026</mark>.
</p>
```

Browsers normally display marked text with a yellow background. Its appearance can be changed using CSS:

```
mark {
  background-color: lightgreen;
  color: black;
}
```

`<mark>` indicates contextual relevance, such as highlighting a search result; it should not replace `<strong>` when indicating importance.

9. `<nav>` Tag

The `<nav>` element groups major navigation links that help users move through a website or webpage.

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="courses.html">Courses</a>
  <a href="contact.html">Contact</a>
</nav>
```

It can also represent page-level navigation:

```
<nav aria-label="Course chapters">
```

```
<a href="#html">HTML</a>
<a href="#css">CSS</a>
<a href="#javascript">JavaScript</a>
</nav>
```

Not every collection of links needs `<nav>`; it is mainly intended for important navigation groups.

10. `<section>` Tag

The `<section>` element groups related content under a common theme. A section should normally include a heading.

```
<section>
  <h2>HTML5 Topics</h2>
  <p>Students will study forms and semantic elements.</p>
</section>

<section>
  <h2>CSS3 Topics</h2>
  <p>Students will study selectors and responsive design.</p>
</section>
```

Each section represents a distinct thematic part of the course information.

Complete Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Technology Blog</title>
</head>
<body>

  <header>
    <h1>Technology Blog</h1>

    <nav aria-label="Main navigation">
      <a href="#home">Home</a>
      <a href="#articles">Articles</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="articles">
      <h2>Latest Articles</h2>

      <article>
        <header>
          <h3>Introduction to HTML5</h3>
```

```

        <p>Published on 27 August 2026</p>
    </header>

    <p>
        HTML5 provides
        <mark>semantic elements</mark>
        that describe webpage structure.
    </p>

    <figure>
        

        <figcaption>
            Figure 1: Semantic structure of an HTML5 page
        </figcaption>
    </figure>

    <aside>
        <h4>Quick Tip</h4>
        <p>
            Use semantic elements to improve readability
            and accessibility.
        </p>
    </aside>

    <footer>
        <p>Written by Prof. Ashima</p>
    </footer>
</article>
</section>
</main>

<footer id="contact">
    <p>&copy; 2026 Technology Blog</p>
</footer>

</body>
</html>

```

Additional HTML5 Tags with Examples

1. <details> Tag

The <details> element creates a section that users can expand or collapse. It is commonly used for FAQs, additional explanations, instructions, and disclosure widgets.

```

<details>
    <summary>View course details</summary>
    <p>This course covers HTML5, CSS3 and JavaScript.</p>
</details>

```

By default, the content is collapsed. Use the open attribute to display it initially:

```

<details open>

```

```
<summary>Important instructions</summary>
<p>Submit the assignment before 30 August 2026.</p>
</details>
```

2. <summary> Tag

The <summary> element provides the visible, clickable heading for a <details> element.

```
<details>
  <summary>What is HTML5?</summary>
  <p>
    HTML5 is the modern standard used to structure
    content on webpages.
  </p>
</details>
```

The <summary> element should be placed as the first child of <details>. Clicking it expands or collapses the remaining content.

3. <time> Tag

The <time> element represents a specific date, time, or duration. The `datetime` attribute provides a machine-readable value for browsers, search engines, and applications.

Date example

```
<p>
  The examination will be held on
  <time datetime="2026-09-15">15 September 2026</time>.
</p>
```

Date and time example

```
<p>
  The webinar begins at
  <time datetime="2026-09-15T10:30:00+05:30">
    10:30 AM on 15 September 2026
  </time>.
</p>
```

The text is readable by users, while the `datetime` value can be processed by software.

4. <bdi> Tag

BDI stands for **Bidirectional Isolation**. The <bdi> element isolates text whose writing direction may be different from the surrounding content.

It is useful when displaying user-generated content containing languages written from right to left.

```
<ul>
  <li>User <bdi>Ashima</bdi>: 95 marks</li>
  <li>User <bdi>علي</bdi>: 90 marks</li>
</ul>
```

The Arabic name has a right-to-left direction. `<bdi>` prevents its direction from disturbing the layout of the surrounding English text.

It is especially useful for:

- Usernames
- Comments
- Multilingual data
- Dynamically generated content

5. `<wbr>` Tag

`WBR` stands for **Word Break Opportunity**. It identifies a position where the browser may break a long word or string if sufficient horizontal space is unavailable.

```
<p>
  Visit:
  https://www.example.com/courses/<wbr>computer-science/<wbr>
  frontend-development
</p>
```

If space is available, the text remains on one line. If space is insufficient, the browser may insert a line break at a `<wbr>` position.

Unlike `<br>`, `<wbr>` does not force a line break.

```
<p>
  Long word:
  WebDevelopment<wbr>AndResponsiveDesign
</p>
```

6. `<datalist>` Tag

The `<datalist>` element provides predefined suggestions for an `<input>` field. It is connected to the input through the `list` attribute.

```
<label for="course">Select or enter a course:</label>

<input type="text"
  id="course"
  name="course"
  list="course-list">

<datalist id="course-list">
  <option value="Computer Science">
```

```
<option value="Information Technology">
<option value="Electronics">
<option value="Mechanical Engineering">
</datalist>
```

The value of the input's `list` attribute must match the `<datalist>` element's `id`.

Users can:

- Select one of the suggested values
- Enter a different value manually

Therefore, `<datalist>` is different from `<select>`, which generally restricts users to the provided options.

7. `<keygen>` Tag — Deprecated

The `<keygen>` element was introduced to generate a public-private key pair for certificate-based authentication.

Historical example:

```
<form action="/register" method="POST">
  <label>Generate security key:</label>
  <keygen name="securityKey">

  <button type="submit">Register</button>
</form>
```

However, `<keygen>` is deprecated and has been removed from modern HTML standards. Modern browsers generally do not support it.

Current web applications should use modern security mechanisms such as:

- Web Authentication API, or WebAuthn
- Passkeys
- TLS client certificates
- Server-managed authentication
- OAuth or OpenID Connect

Therefore, `<keygen>` should be studied only as a deprecated historical HTML element and should not be used in new websites.

8. `<output>` Tag

The `<output>` element displays the result of a calculation or user action.

Addition example

```
<form oninput="result.value =  
    Number(first.value) + Number(second.value)">  
  
    <input type="number" id="first" value="10">  
    +  
    <input type="number" id="second" value="20">  
    =  
  
    <output name="result"  
        for="first second">30</output>  
</form>
```

Here:

- `oninput` performs the calculation whenever a value changes.
- `<output>` displays the result.
- The `for` attribute identifies the controls involved in the calculation.

Range example

```
<form oninput="result.value = volume.value">  
    <label for="volume">Volume:</label>  
  
    <input type="range"  
        id="volume"  
        min="0"  
        max="100"  
        value="50">  
  
    <output name="result" for="volume">50</output>  
</form>
```

9. `<progress>` Tag

The `<progress>` element represents the completion progress of a task.

```
<label for="upload">File upload progress:</label>  
  
<progress id="upload"  
    value="70"  
    max="100">  
    70%  
</progress>
```

Here:

- `value="70"` represents the completed amount.
- `max="100"` represents the total amount.
- The task is 70% complete.

Indeterminate progress

When the completion amount is unknown, omit the `value` attribute:

```
<label for="processing">Processing:</label>
<progress id="processing"></progress>
```

The `<progress>` element is suitable for:

- File uploads
- Downloads
- Form completion
- Installation progress
- Task processing

It should not be used to represent a general measurement, such as temperature or disk usage. The `<meter>` element is more appropriate for such measurements.

10. `<svg>` Tag

SVG stands for **Scalable Vector Graphics**. The `<svg>` element creates vector-based graphics using XML-like elements.

Circle example

```
<svg width="200"
    height="150"
    aria-label="A blue circle">

    <circle cx="100"
        cy="75"
        r="50"
        fill="blue"
        stroke="black"
        stroke-width="3">
    </circle>
</svg>
```

Here:

- `cx` specifies the circle's horizontal centre.
- `cy` specifies its vertical centre.
- `r` specifies its radius.
- `fill` specifies its inner colour.
- `stroke` specifies its border colour.

Rectangle example

```
<svg width="250" height="120">
    <rect x="20"
        y="20"
        width="200"
        height="80"
        fill="lightgreen"
        stroke="darkgreen"
        stroke-width="4">
    </rect>
```

</svg>

SVG graphics can be enlarged without losing quality, making them suitable for:

- Logos
 - Icons
 - Diagrams
 - Charts
 - Illustrations
-

11. <canvas> Tag

The <canvas> element creates a drawing area. JavaScript is required to draw graphics inside it.

Rectangle example

```
<canvas id="drawingArea"
        width="300"
        height="180"
        style="border: 1px solid black;">
  Your browser does not support the canvas element.
</canvas>
```

```
<script>
  const canvas = document.getElementById("drawingArea");
  const context = canvas.getContext("2d");

  context.fillStyle = "blue";
  context.fillRect(50, 40, 200, 100);
</script>
```

Here:

- <canvas> defines the drawing area.
- getContext("2d") creates a two-dimensional drawing context.
- fillStyle sets the colour.
- fillRect() draws a filled rectangle.

Circle example

```
<canvas id="circleCanvas"
        width="250"
        height="200">
</canvas>
```

```
<script>
  const canvas = document.getElementById("circleCanvas");
  const context = canvas.getContext("2d");

  context.beginPath();
  context.arc(125, 100, 60, 0, 2 * Math.PI);
  context.fillStyle = "orange";
  context.fill();
</script>
```

```
context.stroke();
</script>
```

Canvas is commonly used for:

- Games
- Animations
- Dynamic charts
- Image manipulation
- Drawing applications

SVG versus Canvas

Feature	SVG	Canvas
Graphic type	Vector-based	Pixel-based
Creation	HTML/XML elements	JavaScript drawing commands
Scaling	Maintains quality	May become blurred
Element access	Each shape is a DOM element	Drawing is treated as one bitmap
Interactivity	Easy to attach events to shapes	Requires coordinate-based handling
Best suited for	Icons, logos and diagrams	Games, animation and dynamic graphics

HTML5 Multimedia Tags with Examples

1. <audio> Tag

The <audio> element embeds sound content directly into a webpage, such as music, podcasts, lectures, and sound effects.

```
<audio controls>
  <source src="lecture.mp3" type="audio/mpeg">
  <source src="lecture.ogg" type="audio/ogg">

  Your browser does not support the audio element.
</audio>
```

The browser selects the first audio format it supports. The text inside <audio> is displayed only when the browser does not support the element.

Common attributes

Attribute	Purpose
controls	Displays play, pause, volume and progress controls
autoplay	Starts the audio automatically when permitted
loop	Plays the audio repeatedly

Attribute	Purpose
<code>muted</code>	Starts the audio without sound
<code>preload</code>	Suggests how the browser should load the audio
<pre><audio src="background-music.mp3" controls loop muted> </audio></pre>	

Modern browsers may block autoplay when the audio is not muted.

2. `<embed>` Tag

The `<embed>` element inserts an external resource into a webpage. It is commonly used for PDF documents, multimedia, and other browser-supported content.

PDF example

```
<embed src="course-outline.pdf"
      type="application/pdf"
      width="100%"
      height="500">
```

Image example

```
<embed src="campus-map.svg"
      type="image/svg+xml"
      width="600"
      height="400">
```

Common attributes include:

- `src`: specifies the resource location
- `type`: specifies the resource's MIME type
- `width`: specifies its width
- `height`: specifies its height

`<embed>` is a void element, so it does not require a closing tag.

The browser must support the embedded file type. For general webpage embedding, `<iframe>` may sometimes be more appropriate.

3. `<source>` Tag

The `<source>` element defines alternative media files for `<audio>`, `<video>`, and `<picture>` elements.

It allows the browser to choose a supported file format.

Audio example

```
<audio controls>
  <source src="podcast.mp3" type="audio/mpeg">
  <source src="podcast.ogg" type="audio/ogg">

  Your browser does not support audio playback.
</audio>
```

Video example

```
<video width="640" height="360" controls>
  <source src="lecture.mp4" type="video/mp4">
  <source src="lecture.webm" type="video/webm">

  Your browser does not support video playback.
</video>
```

The browser checks the sources from top to bottom and selects the first format it supports.

The `<source>` element does not have a closing tag.

4. `<track>` Tag

The `<track>` element provides text tracks for audio or video, including subtitles, captions, descriptions, and chapters.

It is normally placed inside a `<video>` or `<audio>` element.

```
<video width="640" controls>
  <source src="html-lecture.mp4" type="video/mp4">

  <track src="subtitles-en.vtt"
        kind="subtitles"
        srclang="en"
        label="English"
        default>

  <track src="subtitles-hi.vtt"
        kind="subtitles"
        srclang="hi"
        label="Hindi">
</video>
```

The track file normally uses the **WebVTT** format and has the `.vtt` extension.

Example subtitles-en.vtt file:

WEBVTT

```
00:00:00.000 --> 00:00:04.000
Welcome to the HTML5 lecture.
```

```
00:00:04.500 --> 00:00:08.000
```

Today, we will study multimedia elements.

Important attributes

Attribute	Purpose
<code>src</code>	Specifies the track file
<code>kind</code>	Specifies the type of text track
<code>srclang</code>	Specifies the track language
<code>label</code>	Provides a user-readable title
<code>default</code>	Selects the track initially

Values of `kind`

- `subtitles`: translates spoken dialogue
- `captions`: includes dialogue and important sound information
- `descriptions`: provides descriptions for visually impaired users
- `chapters`: defines navigation points
- `metadata`: provides information for scripts

The `<track>` element is a void element and does not require a closing tag.

5. `<video>` Tag

The `<video>` element embeds video content directly into a webpage without requiring a separate media plugin.

```
<video width="640"
       height="360"
       controls
       poster="lecture-thumbnail.jpg">

  <source src="lecture.mp4" type="video/mp4">
  <source src="lecture.webm" type="video/webm">

  <track src="captions.vtt"
        kind="captions"
        srclang="en"
        label="English"
        default>
```

```
  Your browser does not support the video element.
</video>
```

Common attributes

Attribute	Purpose
<code>controls</code>	Displays video controls

Attribute	Purpose
<code>autoplay</code>	Starts playback automatically when permitted
<code>loop</code>	Repeats the video
<code>muted</code>	Starts the video without sound
<code>poster</code>	Displays an image before playback
<code>width</code>	Specifies the video width
<code>height</code>	Specifies the video height
<code>preload</code>	Suggests how much content should be loaded

Autoplay example

```
<video width="500"
      autoplay
      muted
      loop>
  <source src="campus-tour.mp4" type="video/mp4">
</video>
```

The `muted` attribute is generally required because browsers commonly block videos that attempt to autoplay with sound.

Complete Multimedia Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>Multimedia Learning Portal</title>
</head>
<body>

  <h1>Web Development Resources</h1>

  <section>
    <h2>Audio Lecture</h2>

    <audio controls>
      <source src="html-lecture.mp3"
              type="audio/mpeg">
      <source src="html-lecture.ogg"
              type="audio/ogg">

      Your browser does not support audio.
    </audio>
  </section>

  <section>
    <h2>Video Lecture</h2>
```

```
<video width="640"
        controls
        poster="html-thumbnail.jpg">

    <source src="html-lecture.mp4"
            type="video/mp4">

    <source src="html-lecture.webm"
            type="video/webm">

    <track src="captions-en.vtt"
            kind="captions"
            srclang="en"
            label="English"
            default>

    Your browser does not support video.
</video>
</section>

<section>
    <h2>Course Outline</h2>

    <embed src="course-outline.pdf"
            type="application/pdf"
            width="100%"
            height="500">
</section>

</body>
</html>
```